

 inclusionAI / **Ling-3.0-flash-dspark** 

 like 13

Follow  inclusionAI 2.73k

 Text Generation
  Transformers
  Safetensors
  qwen3
  speculative-decoding
  >> dspark

>> dflash
  specforge
  sglang
  text-generation-inference
  License: other


 Deploy
  → Copy to bucket **NEW**
 Use this model

 **Model card**
 Files
  xet
  Community 2

Ling3-DSpark

A DSpark speculator for Ling3. DSpark extends DFlash with target-model auxiliary features and a confidence head that dynamically chooses the number of draft tokens. The model was trained with SpecForge and is served with SGLang.

Model specifications

- Target model: Ling-3.0-flash
- Draft parameters: 1,363,707,905 (1.36B)
- Draft weight dtype: BF16
- Hidden size: 2,560
- Transformer layers: 5 full-attention layers
- Attention: MHA with 32 query heads and 32 key/value heads
- Target auxiliary feature layers: 1, 11, 23, 29, 35
- Confidence head: vanilla Markov head, rank 256
- DSpark block size: 8 draft tokens (verify width 9, including the target bonus token)

Downloads last month
1,768



Safetensors

Model size 1B params

Tensor type BF16  Files info

Inference Providers **NEW**

 Text Generation

This model isn't deployed by any Inference Provider.

 Ask for provider support

Collection including inclusionAI/...

Ling 3.0  Collection

Lin... • 14 items • Updated 2 ... •  17

- Maximum position embeddings: 262,144

Acceptance length

Acceptance length is the mean number of tokens accepted per speculative verification step, including the target bonus token.

Workload	Acceptance length
GSM8K	6.40
MATH-500	6.29
AIME 2025	5.56
HumanEval	6.57
MBPP	6.34
LiveCodeBench	5.33
MT-Bench	3.92
Alpaca	3.51
Arena-Hard-v2	3.72

The macro mean across the nine workload means is **5.29**.

Serving with SGLang

Launch recipes for this draft on every supported hardware/quantization cell — including the required `--linear-replayssm-cache-len` sizing — with measured speed and accuracy, are in the [SGLang Ling-3.0-flash cookbook](#).

Use an SGLang version with DSPARK support.

Replace the model paths and tensor-parallel size with values appropriate for your deployment:

```
sglang serve \  
  --trust-remote-code \  
  --model-path <LING3_MODEL_PATH> \  
  --tp-size <TP_SIZE> \  
  --speculative-algorithm DSPARK \  
  --speculative-draft-model-path <LING3_DSPARK_MODEL_PATH> \  
  .....
```

Serving with llama.cpp

Use a llama.cpp build with DSpark support. Replace the model paths, quantization type, and GPU layer counts with values appropriate for your deployment. First convert and quantize the target model:

```
python convert_hf_to_gguf.py path/to/Ling-3.0-flash-bf16.gguf \  
  --outfile path/to/Ling-3.0-flash-bf16.gguf \  
  --outtype bf16 --model-name Ling-3.0-flash-bf16
```

```
llama-quantize \  
  path/to/Ling-3.0-flash-bf16.gguf \  
  path/to/Ling-3.0-flash-Q4_K_M.gguf \  
  Q4_K_M
```

Then generate the DSpark draft GGUF:

```
python convert_hf_to_gguf.py \  
  path/to/Ling-3.0-flash-dspark \  
  --target-model-dir path/to/Ling-3.0-flash-dspark \  
  --outtype bf16 \  
  --outfile path/to/Ling-3.0-flash-DSpark-draft.gguf
```

Finally, launch the server with the DSpark draft as the speculative model:

```
llama-server \
  --model path/to/Ling-3.0-flash-Q4_K_M.gguf
  --spec-draft-model path/to/Ling-3.0-flash-Q4_K_M.gguf
  --spec-type draft-dspark --spec-draft-n-its 10
  -ngl all -ngld all -fa on
```